DevOps and Linux Fundamentals

1. Why the Software Industry Needed DevOps

Before defining DevOps, it is important to understand the problem it was created to solve.

Software delivery in the early web

Early websites were mostly collections of static files:

- HTML pages
- CSS stylesheets
- images
- small amounts of JavaScript

A developer created these files and placed them on a web server. Updates were relatively infrequent, and deployment often meant copying a few files to that server.

Operations still existed. Organizations still needed servers, networks and system administrators. However, the software-delivery problem was much smaller than it is today.

Modern applications are continuously changing systems

Applications such as Amazon, Netflix, Instagram, Swiggy and online banking platforms are very different. When a user opens one of these applications:

- identity may be verified;
- data is fetched from databases;
- several services communicate with one another;
- recommendations may be generated;
- payments may be processed;

- logs and metrics are collected;
- security rules are evaluated;
- infrastructure may scale automatically when traffic changes;
- failures must be detected and handled without taking down the complete application.

At the same time, multiple teams are continuously changing the system. One team may add a payment option, another may fix a security vulnerability, and another may modify the database.

> The central challenge of modern software delivery is to keep an application running reliably while it is continuously being changed.

Modern software is rarely “finished.” It is continuously planned, developed, tested, released, operated, monitored and improved.

---

2. Development and Operations

The word **DevOps** combines:

```
Development + Operations
```

Development responsibilities

Development teams commonly:

- write code;
- add features;
- fix defects;
- review and maintain the application.

Operations responsibilities

Operations teams commonly:

- prepare and maintain servers;

- deploy applications;
- manage operating systems and networks;
- configure production environments;
- monitor applications;
- create backups;
- handle incidents;
- maintain availability, security and reliability.

The traditional conflict

Historically, many organizations treated development and operations as separate stages. Developers completed the code and handed it to operations for deployment.

This created conflicting incentives:

```
Development was rewarded for changing the system quickly.
Operations was rewarded for keeping the system stable.
```

A deployment might work in a developer’s environment but fail in production because of:

- a missing dependency;
- a different configuration;
- an incompatible software version;
- an undocumented resource requirement;
- differences between testing and production environments.

Releases therefore became large, manual and risky. Deployment instructions often looked like this:

```
Copy this file.
Stop this service.
Change this configuration.
```

```
Run this database script.
Restart the server.
```

When something failed, each team could blame another part of the system. The organization might have skilled people, but the overall delivery process remained slow and unreliable.

---

3. The Siloed Working Model

A **silo** exists when a team optimizes its own work without sufficient understanding of, communication with or responsibility for the complete outcome.

```
Development team
        ↓
Source code handover
        ↓
Testing team
        ↓
Build handover
        ↓
Operations team
        ↓
Production
```

Every handover can introduce:

- waiting time;
- incomplete information;
- environment differences;
- unclear ownership;
- delayed feedback;
- blame during failures.

Development and operations are not unrelated activities. They form one continuous system:

```
Plan → Code → Build → Test → Release → Deploy → Operate → Monitor
 ↑                                                          ↓
```

```
└──────────────────────── Feedback ────────────────────────┘
```

4. What Is DevOps?

> DevOps is an organizational and technical approach in which the people who build software collaborate closely with the people who run it, automate repeatable work, share responsibility for production outcomes and use continuous feedback to improve the complete software-delivery system.

DevOps asks a practical question:

> How can we deliver software quickly without sacrificing stability and reliability?

DevOps is a philosophy and a way of working

DevOps begins with the idea that development and operations are parts of one delivery system, not separate worlds connected by a final handover.

The developer’s responsibility does not completely end when code is merged. Operations does not become involved only after deployment. Development, testing, security and operations contribute throughout the application’s lifecycle.

DevOps is not a single tool

Git, Jenkins, Docker, Kubernetes and cloud platforms can support DevOps practices, but none of them is individually “DevOps.”

Buying tools without improving ownership, communication and feedback can simply automate a poor process.

DevOps is not only automation

Automation is essential, but DevOps also includes:

- culture;
- collaboration;
- shared responsibility;
- measurement;

- feedback;
- continuous improvement.

DevOps can also be a job title

As systems became more complex, organizations needed specialists who understood areas such as:

- Linux and networking;
- cloud infrastructure;
- application delivery;
- CI/CD pipelines;
- containers and orchestration;
- automation;
- monitoring and observability;
- reliability, security and incident response.

The industry commonly calls such specialists **DevOps Engineers**.

A DevOps engineer does not replace developers or operations teams. The role usually helps create the platforms, automation and practices through which those teams can deliver and operate software effectively.

---

5. Primary Goals of DevOps

5.1 Faster delivery

Useful changes should reach users without unnecessary waiting. Smaller and more frequent releases help an organization:

- deliver value earlier;
- respond to changing requirements;
- fix defects sooner;
- release security patches quickly;

- learn from real user feedback.

5.2 Reliable delivery

Speed without safety can cause broken releases, outages, data loss, dissatisfied users and emergency rollbacks.

DevOps therefore aims to improve **speed and stability together**.

5.3 Repeatability

A critical process should not depend on one engineer remembering 25 manual commands. A documented and automated process is easier to repeat, inspect and improve.

5.4 Fast recovery

Failures cannot always be eliminated. A mature delivery system helps teams detect problems quickly, understand their cause and restore normal service safely.

---

6. Core DevOps Principles

6.1 Collaboration

Development, operations, testing, security and business teams exchange information throughout the delivery lifecycle.

6.2 Shared ownership

The people who build a system also care about how it behaves in production. When a failure occurs, the goal is to improve the system—not merely identify someone to blame.

6.3 Automation

Computers should perform repeatable and deterministic work wherever automation is safe and valuable.

Common areas include:

- compiling code;

- running tests;
- checking code quality and security;
- packaging applications;
- provisioning infrastructure;
- deploying applications;
- checking application health.

6.4 Small and frequent changes

If 100 changes are released together and the release fails, the cause may be difficult to identify. A small change is easier to test, observe, troubleshoot and reverse.

6.5 Fast feedback

Teams should discover problems as early as possible through:

- automated tests;
- build and deployment checks;
- logs, metrics and traces;
- production monitoring;
- user feedback.

6.6 Continuous delivery

“Continuous” does not necessarily mean deploying every second. It means keeping the software in a state in which changes can move toward production frequently and predictably.

The objective is to make deployment a routine operation instead of a rare and dangerous event.

6.7 Infrastructure as Code

With manual configuration, an engineer may install software, create users, open ports, set environment variables and configure services directly on a server. The final state exists, but it may be difficult to reproduce exactly.

Infrastructure as Code (IaC) represents infrastructure and configuration in machine-readable files, normally stored in version control. This makes environments more repeatable, reviewable and traceable.

6.8 Continuous improvement

DevOps is not a final state that an organization reaches once. Teams continuously identify bottlenecks, repetitive work, communication gaps and sources of failure, then improve them.

CALMS Framework

The CALMS framework helps evaluate whether DevOps practices exist across an organization rather than judging adoption by the tools it owns.

Letter	Dimension	Central question
C	Culture	Do teams collaborate and share responsibility?
A	Automation	Is repetitive work performed consistently by automated systems?
L	Lean	Does work flow in small batches with minimal waste and delay?
M	Measurement	Are outcomes measured so that improvement is evidence-based?
S	Sharing	Are knowledge, feedback and lessons shared across teams?

C — Culture

Culture is the collection of values, behaviours and assumptions that shape how people work together.

A siloed culture may assume:

```
Developers own code.
Testers own quality.
Operations owns production.
Security owns security.
```

A DevOps culture instead treats quality, security, reliability and delivery as shared concerns. It encourages psychological safety, open discussion of failures and responsibility for the end result.

A — Automation

Manual delivery processes vary because people can forget steps, use different commands or apply configuration inconsistently.

Automation turns repeatable work into a defined workflow. Good automation should be:

- version-controlled;
- repeatable;
- observable;
- testable;
- safe to rerun where possible.

Automation supports people; it does not remove the need for judgment.

L — Lean

Lean thinking aims to maximize value delivered to the customer while minimizing waste.

In software, activity is not the same as value. A completed feature that waits 40 days for testing and another 20 days for deployment has not yet delivered value to the user.

Common forms of waste include:

- long approval queues;
- large batches of unfinished work;
- repeated manual steps;
- avoidable handoffs;
- unused features;
- rework caused by late feedback.

Small batches and shorter feedback loops help work flow through the system more safely.

M — Measurement

Without measurement, improvement becomes opinion. Teams need evidence to answer questions such as:

- Are deployments becoming more frequent?
- Is delivery time decreasing?
- Are fewer deployments failing?
- Are incidents resolved faster?
- Is reliability improving?

Metrics should guide learning and improvement, not become targets that teams are pressured to manipulate.

S — Sharing

Knowledge should not remain with one person or team. Sharing includes:

- documenting operational knowledge;
- sharing dashboards and production feedback;
- conducting blameless incident reviews;
- reusing tools and delivery patterns;
- communicating lessons from failures;
- helping teams learn from one another.

Sharing reduces dependency on individual experts and strengthens collective ownership.

DORA Software-Delivery Metrics

DORA originally stood for **DevOps Research and Assessment**. Its metrics help teams evaluate software-delivery performance using measurable outcomes rather

than subjective claims.

The original model became widely known through four key metrics:

1. Deployment Frequency
2. Lead Time for Changes
3. Change Failure Rate
4. Mean Time to Restore/Recover (MTTR)

Current DORA guidance uses a five-metric model. It replaces the broad MTTR label with **Failed Deployment Recovery Time** and adds **Deployment Rework Rate**.

Category	Current metric	What it measures
Throughput	Change Lead Time	How quickly a committed change reaches production
Throughput	Deployment Frequency	How often changes are deployed to production
Throughput	Failed Deployment Recovery Time	How quickly the team recovers from a failed deployment
Instability	Change Fail Rate	How often deployments require immediate intervention
Instability	Deployment Rework Rate	How often unplanned deployments are needed because of production incidents

> The metrics should be studied together. High delivery speed is not useful if changes repeatedly fail, while perfect stability achieved by never deploying is also not useful.

1. Deployment Frequency

Deployment Frequency measures how often a team successfully deploys changes to production.

A deployment may include:

- a feature;
- a bug fix;

- a configuration change;
- a security patch;
- a dependency update;
- an infrastructure change.

An organization may measure deployments per day, week or month, or measure the average time between deployments. The definition must remain consistent.

Example

If a team performs 40 successful production deployments during 20 working days:

```
Deployment Frequency = 40 ÷ 20
                     = 2 deployments per working day
```

Frequent deployment can indicate smaller batches, a repeatable delivery process, effective automation and faster feedback. It should not be increased artificially through meaningless or failed deployments.

Deployment versus release

- **Deployment:** software is installed in the production environment.
- **Release:** functionality becomes available to users.

A feature flag can separate the two events:

```
Code deployed to production
            ↓
Feature flag remains OFF
            ↓
Team verifies the deployment
            ↓
Feature enabled for selected users
```

2. Change Lead Time

Change Lead Time measures the time from a code change being committed to version control until it is successfully deployed to production.

```
Commit → Build → Test → Review/Approval → Deployment → Production
```

```
Change Lead Time = Production Deployment Time − Commit Time
```

Shorter lead time enables faster feedback, quicker defect fixes, earlier customer value and faster security updates.

Teams must define the measurement method consistently. For example, they may measure every commit, use the earliest commit in a deployment or measure from pull-request merge to production.

3. Change Fail Rate

Change Fail Rate measures the proportion of production deployments that require immediate intervention, such as a rollback, hotfix, configuration correction or feature disablement.

$$\text{Change Fail Rate} = \frac{\text{Failed Production Deployments}}{\text{Total Production Deployments}} \times 100$$

Example

If 4 of 40 production deployments require immediate remediation:

```
Change Fail Rate = (4 ÷ 40) × 100
                 = 10%
```

The organization should define what counts as a failure. Examples may include:

- a service outage;
- a serious production defect;
- rollback of the deployment;
- an immediate hotfix;

- severe performance degradation;
- disabling the released feature.

Possible ways to reduce the rate include smaller changes, stronger automated tests, production-like environments, feature flags, health checks, canary releases and backward-compatible database changes.

4. Failed Deployment Recovery Time

Failed Deployment Recovery Time measures how long it takes to restore normal service after a deployment fails and requires immediate intervention.

This name is more specific than the older term **MTTR**, which has been used to mean Mean Time to Repair, Recover, Restore or Resolve.

Example

Suppose three deployment-related failures take 20, 40 and 90 minutes to recover from:

```
Average Recovery Time = (20 + 40 + 90) ÷ 3
                      = 50 minutes
```

A shorter recovery time may be supported by:

- clear monitoring and alerts;
- reliable rollback procedures;
- incident-response runbooks;
- small and reversible changes;
- feature flags;
- good observability.

5. Deployment Rework Rate

Deployment Rework Rate measures the proportion of deployments that are unplanned and performed in response to a production incident.

Examples include emergency patches, corrective deployments and incident-driven configuration changes.

This metric helps reveal how much delivery capacity is being consumed by repairing earlier changes rather than delivering planned value.

Using DORA metrics correctly

DORA metrics are most useful when:

- measured for a specific application or service;
- observed as trends over time;
- interpreted in the application's context;
- shared across development, operations and release teams;
- used to identify bottlenecks and guide improvement.

They should not be used as isolated employee-performance targets or for simplistic comparisons between unrelated teams.

Reference: DORA's software-delivery performance metrics

DevOps, SRE and Platform Engineering

These concepts overlap, but each begins with a different primary problem.

Approach	Primary problem	Main focus
DevOps	Development and operations work in silos	Collaboration, shared ownership, automation and continuous delivery
Site Reliability Engineering (SRE)	Production reliability needs an engineering discipline	Reliability objectives, incident response, automation and operational risk
Platform Engineering	Developers face excessive infrastructure complexity	Self-service platforms, reusable paths and a better developer experience

DevOps

DevOps addresses slow handoffs, conflicting goals, unclear ownership and delayed feedback between the people who build and operate software.

It is primarily a philosophy and operating model rather than one fixed organizational structure.

Site Reliability Engineering

SRE applies software-engineering practices to operations and reliability. It helps teams decide:

- how reliable a service must be;
- how reliability should be measured;
- when reliability work should take priority over feature delivery;
- how incidents should be handled;
- which operational tasks should be automated.

SRE is one practical way to implement many DevOps principles, especially for services with demanding availability and scale requirements.

Platform Engineering

As an organization grows, every development team may otherwise need to understand cloud infrastructure, Kubernetes, CI/CD, networking, observability, security policies and secrets management.

Platform engineering creates reusable internal capabilities that developers can consume through self-service interfaces. A platform may provide:

- application templates;
- standard CI/CD pipelines;
- approved infrastructure patterns;
- environment provisioning;
- secrets and configuration management;
- built-in monitoring and security controls.

The goal is to reduce unnecessary cognitive load while preserving safe and standardized delivery.

The Dangerous “DevOps Team” Anti-Pattern

An organization may try to solve silos by adding another handoff:

```
Developers → DevOps team → Operations
```

If developers still submit tickets for every pipeline, deployment and environment change, the new team can become another wall rather than enabling DevOps.

A dedicated DevOps or enablement team can be valuable when it:

- builds shared automation;
- coaches product teams;
- creates reusable delivery patterns;
- improves developer self-service;
- makes production feedback easier to access.

It becomes counterproductive when it is the only team allowed to understand or operate the delivery system.

> A DevOps team should enable shared ownership, not become a new silo.

Linux Essentials

A Linux server can initially be understood as four connected areas:

```
Filesystem  → Where programs and data are stored
Permissions → Who can access those resources
Processes   → Programs currently running
systemd     → Starts, stops and supervises long-running services
```

This introduction focuses on the operating system, kernel, shell and terminal—the foundation needed before working with files, permissions, processes and services.

1. Operating System

Applications cannot safely manage the processor, memory, disk, network devices, files and other programs directly.

The operating system manages these resources and provides controlled services to applications:

```
Applications
     ↓
Operating System
     ↓
Hardware
```

It provides abstractions such as files, processes, users, sockets and virtual memory so every application does not need to understand the hardware in detail.

2. Linux Kernel

Strictly speaking, **Linux is the kernel**. The kernel is the core part of the operating system and runs with enough privilege to manage the computer's resources.

It is responsible for:

- process scheduling;
- memory management;
- filesystem access;
- hardware devices and drivers;
- networking;
- permission enforcement;
- system calls.

The kernel is loaded into memory during startup and normally remains active until shutdown.

How an application uses the kernel

Suppose a Java application wants to read `app.env`:

```
Java application requests the file
              ↓
Runtime and operating-system libraries issue a system call
              ↓
Kernel resolves the path and checks permissions
              ↓
Kernel requests data from the filesystem and storage device
              ↓
Data is returned to the application
```

The application does not directly control the disk. It asks the kernel to perform the protected operation.

---

3. Major Responsibilities of the Kernel

3.1 CPU and process scheduling

A **process** is a running instance of a program. A server may run Nginx, MySQL, a Spring Boot application and a shell at the same time.

The kernel schedules CPU time among runnable processes:

```
Run Nginx briefly
Pause Nginx
Run MySQL
Pause MySQL
Run Spring Boot
Run Bash
```

The switches happen rapidly, creating the experience that many programs are running simultaneously. On multi-core processors, multiple processes or threads can also execute at the same instant on different cores.

3.2 Memory management

The kernel decides:

- which memory a process may use;
- how virtual memory maps to physical memory;
- whether memory may be shared;
- how memory pressure is handled;
- whether a process is attempting unauthorized access.

```
Spring Boot process → its virtual address space
MySQL process       → its virtual address space
Nginx process       → its virtual address space
```

This isolation normally prevents one application from directly reading or corrupting another application's memory.

3.3 Filesystem management

A storage device contains blocks of data. A filesystem organizes that storage into human-usable concepts such as:

- files and directories;
- names and paths;
- metadata;
- ownership;
- permissions.

Examples of Linux paths include:

```
/home/aditya/app.jar
/etc/nginx/nginx.conf
/var/log/application.log
```

When an application asks to open `/etc/app/config.yml`, the kernel helps determine:

1. Does the path exist?
2. Is it a file or a directory?
3. Who owns it?
4. Does the requesting process have permission?
5. Where is the data stored?
6. Which device driver should access it?

3.4 Device management

Applications normally interact with hardware through the kernel and device drivers:

```
Application
    ↓
Kernel
    ↓
Device driver
    ↓
Hardware
```

For example, when an application writes a file, the kernel coordinates the request and the storage driver communicates with the SSD. The application does not need to understand the SSD's hardware protocol.

3.5 Networking

The kernel manages low-level networking tasks such as network interfaces, packets, routing and sockets. Applications use operating-system APIs to send and receive data without controlling the network hardware directly.

---

4. Kernel Space and User Space

Modern operating systems separate execution into two broad privilege areas:

```
User space
Kernel space
```

Kernel space

The kernel runs with very high privilege. It can access physical memory, communicate with hardware, control CPU execution, manage processes and enforce security rules.

Because of this authority, a serious kernel failure can crash the entire operating system.

User space

Most programs run with restricted authority in user space. Examples include:

- Bash and Zsh;
- Java applications;
- Nginx and MySQL;
- Docker CLI;
- `ls` and `curl`;
- Chrome and IntelliJ IDEA.

If a normal user-space application crashes, the operating system usually continues running. The kernel can terminate that process and reclaim its resources.

---

5. System Calls

A **system call** is a controlled entry point through which a user-space program requests a protected operation from the kernel.

Applications use system calls to perform tasks such as:

- opening or reading a file;
- creating a process;
- allocating memory;
- sending network data;
- changing file permissions.

System calls preserve the boundary between restricted applications and the highly privileged kernel.

---

6. Linux Distribution

The Linux kernel alone is not a complete everyday environment. A **Linux distribution** combines the kernel with other components, such as:

- command-line utilities;
- system libraries;
- one or more shells;
- a package manager;
- a service manager;
- default configuration and software repositories.

Examples include:

- Ubuntu;
- Debian;
- Fedora;
- Red Hat Enterprise Linux;
- Rocky Linux;
- Amazon Linux.

Different distributions can use the same Linux kernel while providing different package managers, defaults and release policies.

---

7. Shell

A **shell** is a user-space program that accepts commands, interprets them and starts other programs.

Common shells include:

- Bash;

- Zsh;
- Fish;
- `sh`.

The shell is neither the terminal nor the kernel:

```
User
 ↓
Shell
 ↓
Kernel
 ↓
Hardware
```

What happens when a command is entered?

Suppose the user runs:

```
ls /var/log
```

The shell broadly performs these steps:

1. Reads the entered command.
2. Parses the command and its arguments.
3. Identifies `ls` as the program and `/var/log` as its argument.
4. Finds the executable file for `ls`.
5. Asks the kernel to create a process.
6. Runs `ls` inside that process.
7. Waits for the program to finish.
8. Displays the next prompt.

---

8. Terminal

A **terminal** is the interface through which a user interacts with command-line programs such as a shell. Today, this is normally a terminal emulator.

Examples include:

- macOS Terminal;
- Windows Terminal;
- the IntelliJ terminal panel;
- the VS Code integrated terminal.

The terminal mainly handles:

- keyboard input;
- text display;
- cursor position;
- colours and formatting;
- window dimensions;
- communication with the shell.

The terminal itself does not normally interpret commands such as `ls`, `cd`, `docker` or `java`. The shell interprets them.

```
Terminal = interface and communication channel
Shell    = program that interprets commands
```

Terminal and shell are separate programs

When the macOS Terminal application is opened:

```
Terminal application starts
          ↓
Terminal starts a shell, such as Zsh
          ↓
The shell displays a prompt
          ↓
The user enters commands
```

The same terminal window can run another shell:

```
bash
```

Running `exit` closes that Bash process and returns to the previous shell.

9. Complete Command-Execution Flow

When the user runs `ls /var/log`, the complete conceptual flow is:

```
Terminal receives the user's typing
                  ↓
Shell parses the command
                  ↓
Shell locates the ls executable using PATH
                  ↓
Kernel creates a process for ls
                  ↓
ls requests the directory contents from the kernel
                  ↓
Kernel checks permissions and reads filesystem metadata
                  ↓
ls writes the result to the terminal
```

This single command connects the major ideas introduced in this chapter:

- the terminal provides the interface;
- the shell interprets the command;
- the kernel creates and manages the process;
- the filesystem organizes the data;
- permissions determine whether access is allowed;
- the terminal displays the output.